



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_COMPILER DESIGN

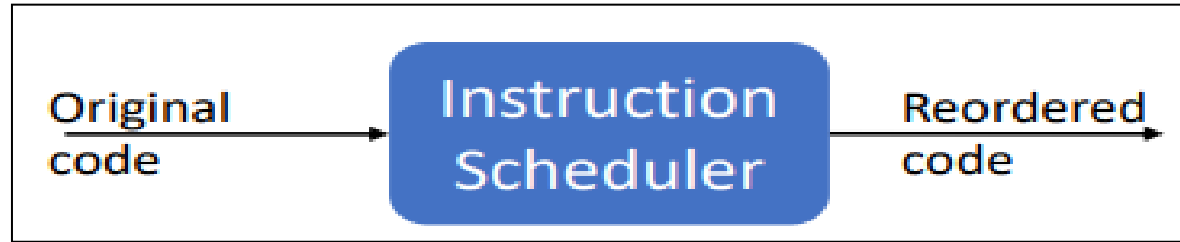


PARALLELISM

Dr. B.V. Baiju,
SCOPE,Assistant Professor
VIT, Vellore

Instruction Scheduling

- The aim of instruction scheduling is to find a valid order that optimizes some metric, like execution speed.



- Scheduler assumes
 - ***a fixed set of operations and does not rewrite code.***
 - ***a fixed allocation of values to registers***
 - May rename registers but does not change allocation decisions

```
LD A, R1
LD B, R2
R3 = R1 + R2
LD C, R4
R5 = R4 * R2
R6 = R3 + R5
ST R6, D
```

```
LD C, R4
LD B, R2
LD A, R1
R5 = R4 * R2
R3 = R1 + R2
R6 = R3 + R5
ST R6, D
```

Data Dependences

- When parallelizing a program, it is important to determine what data is accessed by which statements, and how these statements depend on each other
- **Flow (True) Dependence** : Variables/registers defined in one instruction are used in a later instruction
- **Anti Dependence** : Variables/registers used in one instruction are overwritten by a later instruction:

```
for(i = 0; i < n; ++i)  
  a[i] = a[i + 1];
```

- A loop where $a[i]$ is modified in iteration i and read in iteration $i - 1$.

- **Output Dependence** : Variables/registers defined in one instruction are overwritten by a later instruction
- *Data dependences prevent instructions from being reordered, or executed at the same time.*
- S1 and S2 can execute in parallel if there are no dependences between S1 and S2 – true dependences – anti-dependences – output dependences Some dependences can be removed.

Example

- Parallelism often occurs in loops.

```
for(i=0; i<100; i++)  
a[i] = i;
```

- No dependences.
- Iterations can be executed in parallel.

```
for(i=0; i<100; i++)  
{  
a[i] = i;  
b[i] = 2*i;  
}
```

- Iterations and statements can be executed in parallel.

```
for(i=0;i<100;i++) a[i] = i;  
for(i=0;i<100;i++) b[i] = 2*i;
```

- Iterations and loops can be executed in parallel.

```
for( i=0; i<100; i++ )  
a[i] = f(a[i-1]);
```

- Dependence between $a[i]$ and $a[i-1]$
- Loop iterations are not parallelizable

Loop-Carried Dependence

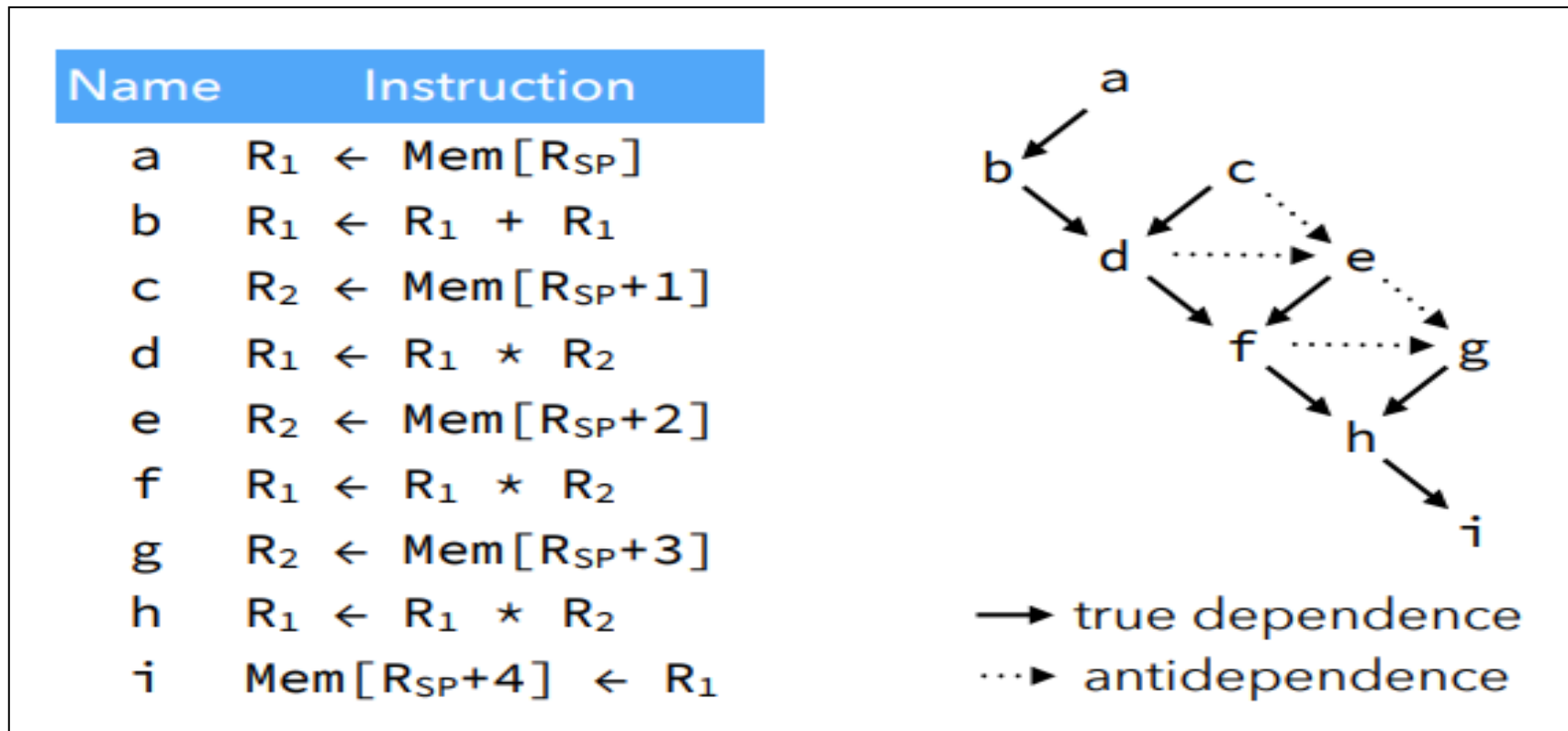
- A **loop-carried dependence** is a dependence that is present *only if the statements occur in two different instances of a loop*
- Otherwise, we call it a loop-independent dependence
- Loop-carried dependences limit loop iteration parallelization

```
for(i=0; i<100; i++ )  
  for(j=0; j<100; j++ )  
    a[i][j] = f(a[i][j-1]);
```

- Loop-independent dependence on i.
- Loop-carried dependence on j.
- Outer loop can be parallelized, inner loop cannot.

Dependence graph

- The dependence graph is a directed graph representing dependences among instructions.
- Its nodes are the instructions to schedule, and there is an edge from node $n1$ to node $n2$ iff the instruction of $n2$ depends on $n1$.
- Any topological sort of the nodes of this graph represents a valid way to schedule the instructions.



Scheduling Example

<u>Operation</u>	<u>Cycles</u>
load	3
loadl	1
loadAl	3
store	3
storeAl	3
add	1
mult	2
fadd	1
fmult	2
shift	1
branch	0 to 8

- **Loads & stores may or may not block**
 - > Non-blocking $\Rightarrow$ fill those issue slots
- **Branches typically have delay slots**
 - > Fill slots with operations unrelated to branch condition evaluation
- **Branch Prediction may hide branch latencies (hardware feature)**

Build a simple local scheduler (basic block)

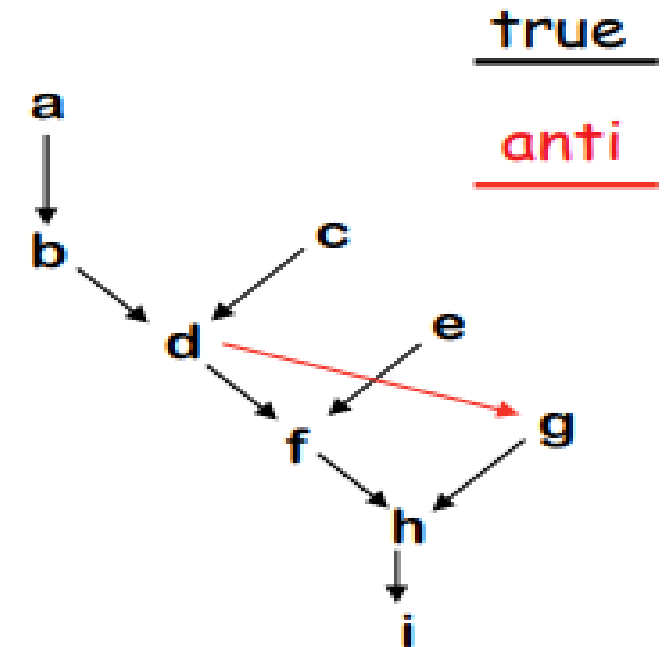
- non-blocking loads & stores
- different latencies load/store vs. arith. etc. operations
- different heuristics
- forward / backward scheduling

1. Build the dependence graph

1	a:	loadAl	r0,@w	⇒ r1
4	b:	add	r1,r1	⇒ r1
5	c:	loadAl	r0,@x	⇒ r2
8	d:	mult	r1,r2	⇒ r1
9	e:	loadAl	r0,@y	⇒ r3
12	f:	mult	r1,r3	⇒ r1
13	g:	loadAl	r0,@z	⇒ r2
16	h:	mult	r1,r2	⇒ r1
18	i:	storeAl	r1	⇒ r0,@w
21				

The Code

⇒ 20
cycles



The Dependence Graph

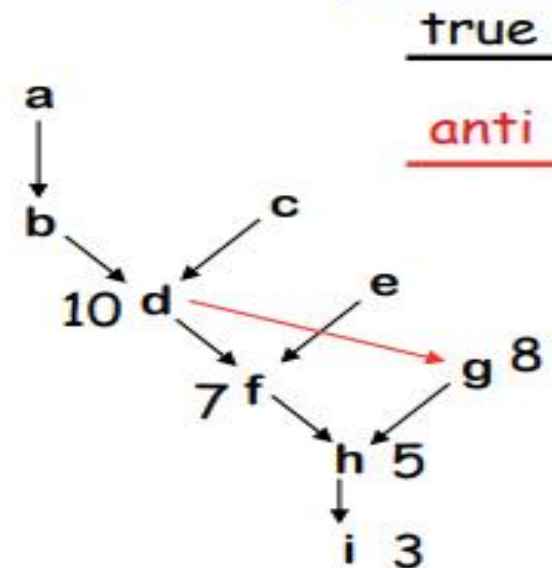
1. Build the dependence graph
2. Determine priorities: longest latency-weighted path

Operation	Cycles
load	3
loadl	1
loadAl	3
storeAl	3
add	1
mult	2

```

a:  loadAl    r0,@w    => r1
b:  add       r1,r1    => r1
c:  loadAl    r0,@x    => r2
d:  mult      r1,r2    => r1
e:  loadAl    r0,@y    => r3
f:  mult      r1,r3    => r1
g:  loadAl    r0,@z    => r2
h:  mult      r1,r2    => r1
i:  storeAl   r1        => r0,@w
  
```

The Code

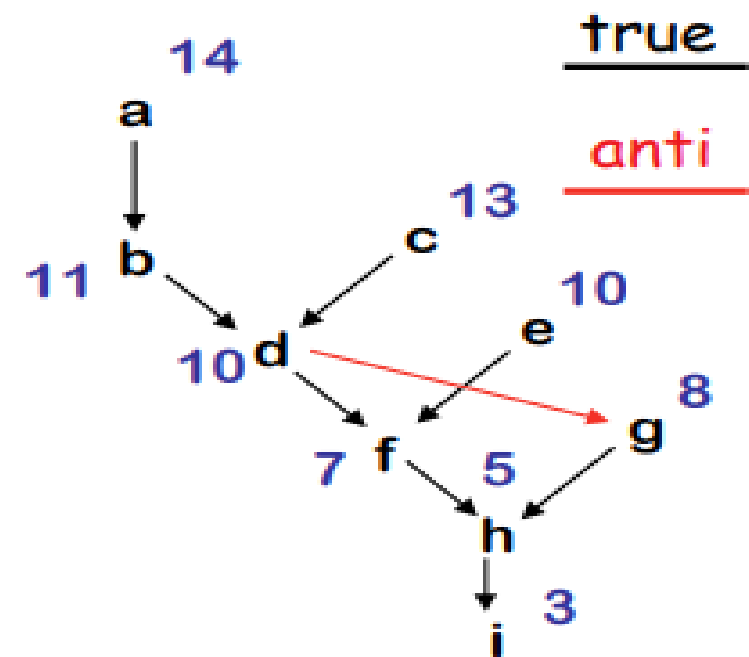


The Dependence Graph

1. Build the dependence graph
2. Determine priorities: longest latency-weighted path

a:	loadAl	r0,@w	⇒ r1
b:	add	r1,r1	⇒ r1
c:	loadAl	r0,@x	⇒ r2
d:	mult	r1,r2	⇒ r1
e:	loadAl	r0,@y	⇒ r3
f:	mult	r1,r3	⇒ r1
g:	loadAl	r0,@z	⇒ r2
h:	mult	r1,r2	⇒ r1
i:	storeAl	r1	⇒ r0,@w

The Code

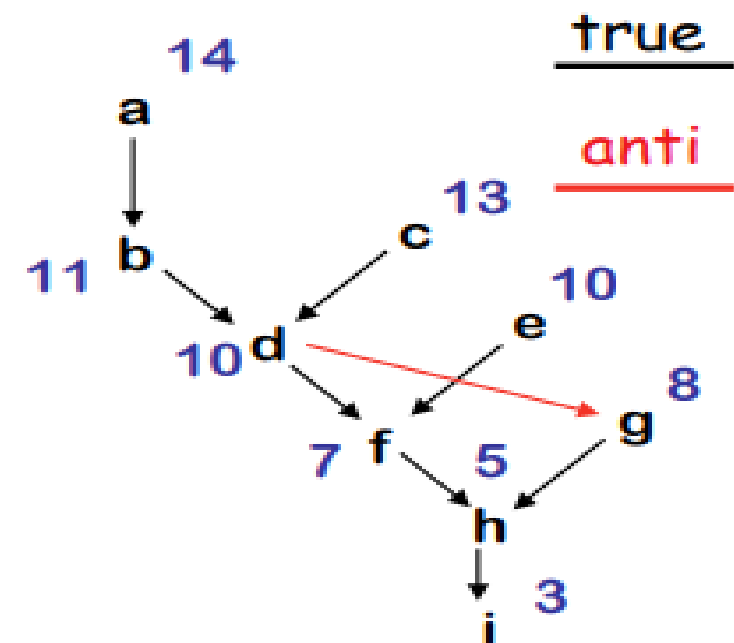


The Dependence Graph

1. Build the dependence graph
2. Determine priorities: longest latency-weighted path

a:	loadAl	r0,@w	⇒ r1
b:	add	r1,r1	⇒ r1
c:	loadAl	r0,@x	⇒ r2
d:	mult	r1,r2	⇒ r1
e:	loadAl	r0,@y	⇒ r3
f:	mult	r1,r3	⇒ r1
g:	loadAl	r0,@z	⇒ r2
h:	mult	r1,r2	⇒ r1
i:	storeAl	r1	⇒ r0,@w

The Code



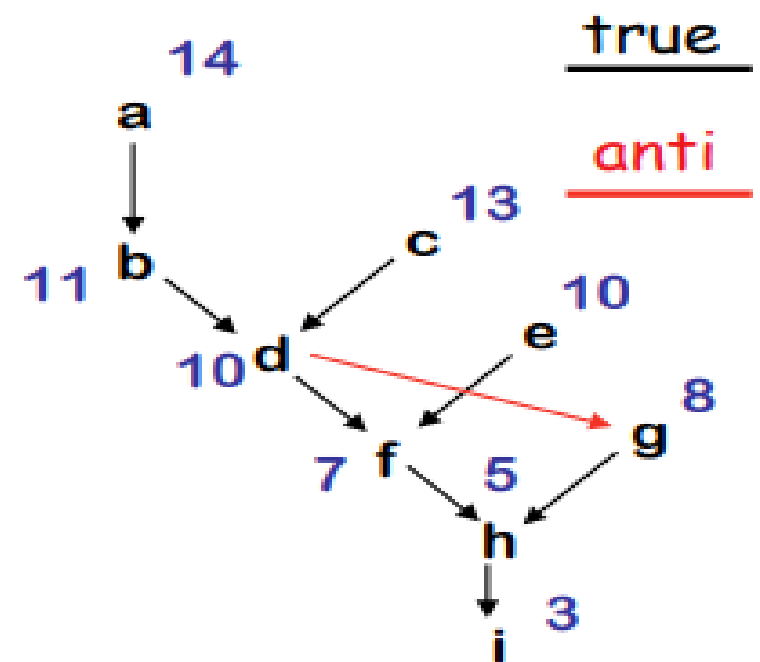
The Dependence Graph

Note: Here we assume that operation has to finish to satisfy an anti dependence.

1. Build the dependence graph
2. Determine priorities: longest latency-weighted path
3. Perform list scheduling (forward)

a:	loadAl	r0,@w	⇒ r1
b:	add	r1,r1	⇒ r1
c:	loadAl	r0,@x	⇒ r2
d:	mult	r1,r2	⇒ r1
e:	loadAl	r0,@y	⇒ r3
f:	mult	r1,r3	⇒ r1
g:	loadAl	r0,@z	⇒ r2
h:	mult	r1,r2	⇒ r1
i:	storeAl	r1	⇒ r0,@w

The Code

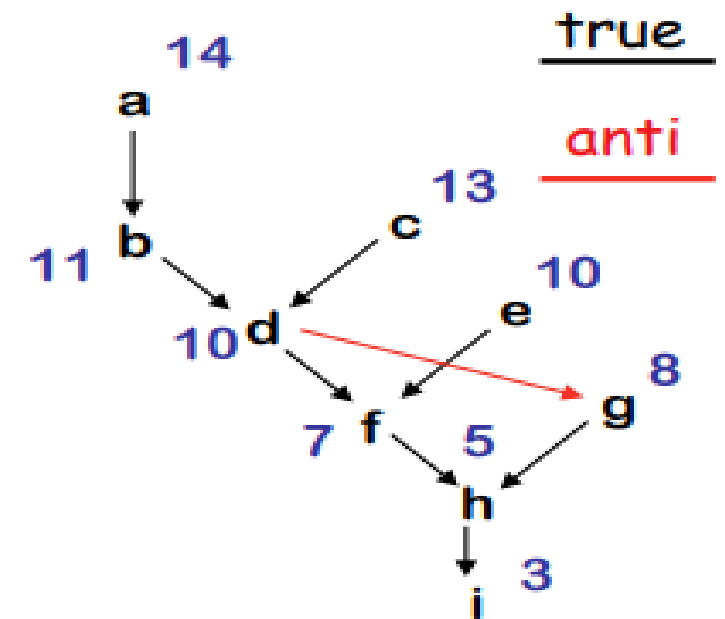


The Dependence Graph

1. Build the dependence graph
2. Determine priorities: longest latency-weighted path
3. Perform list scheduling (forward)

1	a:	loadAl	r0,@w	⇒ r1
2	c:	loadAl	r0,@x	⇒ r2
3	e:	loadAl	r0,@y	⇒ r3
4	b:	add	r1,r1	⇒ r1
5	d:	mult	r1,r2	⇒ r1
7	g:	loadAl	r0,@z	⇒ r2
8	f:	mult	r1,r3	⇒ r1
10	h:	mult	r1,r2	⇒ r1
12	i:	storeAl	r1	⇒ r0,@w
15				

The Code



The Dependence Graph

⇒ 14
cycles

$a = b * c$

$d = e + f$

These instructions do not have any
dependence. Can be executed in parallel

- Other constraints
- Some architectures have more than one ALU
- But what if there is only one ALU?
- Cannot execute in parallel
- If a multiply takes two cycles to complete, cannot even execute the second instruction immediately after the first
- Resource constraints are limitations of the hardware that prevent instructions from executing at a certain time

Software Pipelining

- Software pipelining is a *compile-time scheduling technique* that overlaps subsequent loop iterations to disclose operation-level parallelism.
- It derives its performance gain by filling delays within each iteration of a loop body with instructions from different iterations of that same loop,
- This method requires fewer registers per loop iteration than loop unrolling
- This method requires some extra code to fill (**preheader**) and drain (**postheader**) the software pipelined loop.
- A necessary issue with the development of adequate software pipelining algorithms is how to deal with *loops with conditional branches*.
- Conditional branches raise the complexity and reduce the performance of software pipelining algorithms by offering few possible execution paths into the scheduling opportunity.

- Let us assume a RISC-like intermediate code for the loop body such as

```
load r100, b(i);  
fmul r100, 2.0, r100;  
store a(i), r100;
```

- While demonstrating the principle of software pipelining, we *focus only on the loop body*, neglecting some prologue and epilogue code.
- It can create the following assumptions about the ILP-processor.
 - It has **independent execution units** for FP, FX, load, and store instructions, all adequate of parallel operations.
 - All execution units should allow a **new operation to be initiated** in every cycle.
 - Finally, we suppose that the FP unit delivers the result of the fmul instruction in say, three cycles, whereas load and stores have an execution latency of one cycle.

- Now let us look for the most **parallel execution** feasible.
- We achieve it when we unroll the loop and subsequent iterations in as parallel a fashion as possible. Let us start with the first iteration. It can be executed as follows

Cycle	Instructions	Comment
C	load r101, b(1);	// b(1) is loaded
c+1	fmul r101, 2.0, r101	
c+2	decr r200;	// decrement loop count
c+3	Nop	// wait for result of fmul
c+4	store a(i) +, r101;	// store a(i), autoincrement i.

- The supposed latency of the final operation is three cycles, therefore in cycle $c + 3$ its result is not yet available and a 'nop' has to be inserted.
- Under the assumption made, the second iteration can be initiated in cycle $c+1$ by loading the second data item, $b(2)$.
- It can avoid interference with the first iteration that is to avoid a WAW conflict in using r101, in the second iteration r101 has to be renamed, say to r102.

- Subsequently, *both iterations can be implemented in parallel.*
- This can receive the following execution sequence

Cycle	1.Iteration	2.Iteration
C	load r101, b(1);	
c+1	fmul r101, 2.0, r101,	load r102, b(2);
c+2 r102	decr	fmul r102, 2.0,
c+3	Nop	Decr
c+4	store a(i) +, r101;	Nop
c+5		store a(2) +, r102;

- In the next table, it can show the entire execution of the loop.
- Let us see cycles c+4, c+5, and c+6.
- These are the ones displaying the real advantage of software pipelining.
- The important point is that for these cycles the available parallelism between subsequent loop iterations is completely used.

- For instance, in cycle c+4 the parallel operations are as follows
 - It can be storing the result of iteration 1 (that is, $a(1)$), auto-incrementing the index.
 - It can be decrementing the loop count, which is maintained in r200 by 1.
 - It can be performing the floating-point multiplication with the operands belonging to cycle 4, that is $(2.0 * b(4))$;
 - It can be loading the operand for iteration 5 that is $b(5)$.

Cycle	Iteration Number						
	1	2	3	4	5	6	7
C	load						
c+1	Fmul	load					
c+2	decr	fmul	Load				
c+3	nop	decr	fmul	Load			
c+4	store	Nop	Decr	Fmul	load		
c+5		store	Nop	Decr	fmul	load	
c+6			store	Nop	decr	fmul	Load
c+7				store	nop	decr	fmul
c+8					store	nop	Decr
c+9						store	Nop
c+10							store

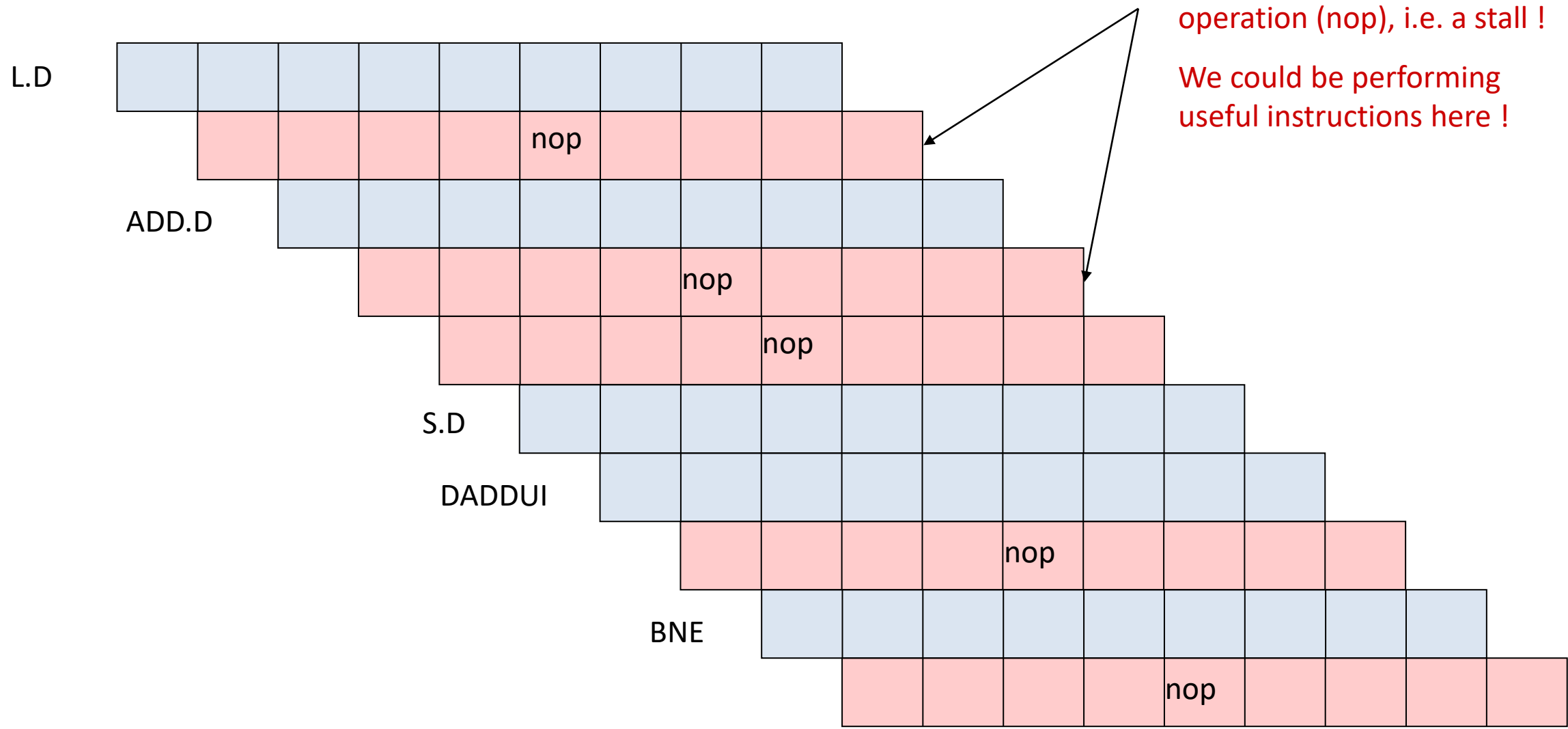
Consider the instruction sequence from before:

```
Loop :  L.D      F0,0(R1)      ; F0 = array elem.
        ADD.D   F4,F0,F2     ; add scalar in F2
        S.D     F4,0(R1)     ; store result
        DADDUI  R1,R1,#-8    ; decrement ptr
        BNE     R1,R2,Loop   ; branch if R1 !=R2
```

This was executed in the following sequence on our pipeline:

Loop :	L.D	F0,0(R1)	1
		stall	2
	ADD.D	F4,F0,F2	3
		stall	4
		stall	5
	S.D	F4,0(R1)	6
	DADDUI	R1,R1,#-8	7
		stall	8
	BNE	R1,R2,Loop	9
		stall	10

A pipeline diagram for the execution sequence is given by:



Software pipelining *eliminates nop's* by inserting instructions from different iterations of the same loop body:

- 1 → unroll loop body with an unroll factor of n. we'll take n = 3 for our example
- 2 → select order of instructions from different iterations to pipeline
- 3 → “paste” instructions from different iterations into the new pipelined loop body

Step 1 → Unroll loop body with an unroll factor of n. we'll take n = 3 for our example

Iteration i:	L.D	F0,0(R1)
	ADD.D	F4,F0,F2
	S.D	F4,0(R1)
Iteration i + 1:	L.D	F0,0(R1)
	ADD.D	F4,F0,F2
	S.D	F4,0(R1)
Iteration i + 2:	L.D	F0,0(R1)
	ADD.D	F4,F0,F2
	S.D	F4,0(R1)

- Notes:
- 1.) We are unrolling the loop body
Hence no loop overhead Instructions are shown!
 - 2.) There three iterations will be “collapsed” into a single loop body containing instructions from different iterations of the original loop body.

Step 2 → Select order of instructions from different iterations to pipeline

Iteration i:

L.D F0,0(R1)

ADD.D F4,F0,F2

S.D F4,0(R1)

Iteration i + 1:

L.D F0,0(R1)

ADD.D F4,F0,F2

S.D F4,0(R1)

Iteration i + 2:

L.D F0,0(R1)

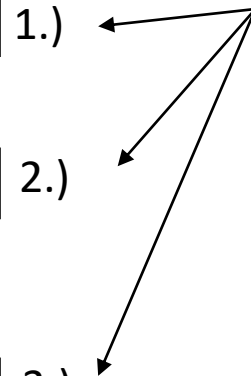
ADD.D F4,F0,F2

S.D F4,0(R1)

Notes:

1.) We'll select the following order in our pipelined loop:

2.) Each instruction (L.D ADD.D S.D) must be selected at least once to make sure that we don't leave out any instructions when we collapse The loop on the left into a single pipelined loop.



Step 3 → “paste” instructions from different iterations into the new pipelined loop body

Iteration i:

L.D F0,0(R1)

ADD.D F4,F0,F2

S.D F4,0(R1)

Iteration i + 1:

L.D F0,0(R1)

ADD.D F4,F0,F2

S.D F4,0(R1)

Iteration i + 2:

L.D F0,0(R1)

ADD.D F4,F0,F2

S.D F4,0(R1)

THE Pipelined Loop

Loop: S.D F4,16(R1) ; M[i]
ADD.D F4,F0,F2 ; M[i - 1]
L.D F0,0(R1) ; M[i - 2]
DADDUI R1,R1,#-8
BNE R1,R2,Loop

1.)

Loop:

2.)

3.)

Now just insert a loop preheader & postheader and the pipelined loop is finished:

